Problem and Context

Our project is a *layered* functional programming language akin to Haskell. The layered nature of the language comes from the ability of users to enable and disable language features, such as algebraic datatypes and parametric polymorphism. This will ideally provide a gentle introduction to functional programming for students in introductory courses. The idea is that students will begin with an extremely basic language variant, and then procedurally work their way up to complex variants over the course of the term.

The layered nature of our language introduces several nontrivial difficulties and design considerations. Specifically, our problems can be summarized by the *expression problem*, which is about how one can extend a software system without modifying or recompiling existing code, while also retaining complete static type safety. These extensions are facilitated in two dimensions: adding new datatypes, and adding new operations. Our layered language will have to do *both*. New datatypes must be added for each language variant, while new function definitions will have to be added within the parser, interpreter, and typechecker. While there are a fair number of solutions to the expression problem, we've yet to decide on a sound solution in the context of our project.

To facilitate this investigation, our team must first make the important decision of *how to represent the core features of the language*. Once we have the representation, we will then be able to define a parser, interpreter, and typechecker. This notion sets the foundation of the system, and is thus quite difficult to reverse once we have settled on an approach. Without agreement, our project cannot realistically continue.

Options

There are several pathways we can take, which differ quite substantially in complexity, elegance, and foreseen maintainability. I will take a procedural approach, opting to discuss the "simplest" options first before moving onto discussions with more formality, and perhaps those that might be outside of the remit of the collective group.

Ad-hoc checking (equivalent to "do nothing")

The simplest and most direct approach is to define a "master" datatype for expressions and types. I equate this with "do nothing" (since an actual "do nothing" would effectively terminate the project) since it is the "obvious" approach. The corresponding datatypes would capture every feature of the language simultaneously, and ad-hoc errors would have to be thrown whenever a disabled feature is encountered.

- *This approach is very simple, almost to the point of being naïve.* However, it may lead to the most understandable implementation, since its facilitation would be straightforward.
- *This approach would be inelegant*. Writing datatypes capturing every possible feature of the language simultaneously would lead to a huge "monolith". Any change to the datatype(s) would then propagate to every single construct in the implementation that referred to it.
- *This approach would be only slightly maintainable.* While this approach, in its directness, would be most understandable at a glance, as mentioned, any significant change would reflect in nontrivial changes across the entire implementation.

Datatypes à la carte

A particularly elegant approach is *datatypes à la carte* (DTALC), in which functors are wrapped in a fixed point combinator.

```
newtype Fix f = In { out :: f (Fix f) }
```

Individual datatypes are then lifted into this functor to facilitate a shared recursive structure.

To combine datatypes, one can define a coproduct on functors, and then write a typeclass hierarchy for injecting constructors into coproduct types. The coproduct is defined as follows.

```
data (f :+: g) x = L (f x) | R (g x)
```

I summarize the benefits and drawbacks as follows.

- *Even without full understanding, it's evident that this approach is quite complex*. Understanding the mechanics behind the approach is quite nontrivial, even if one does not need to inherently understand the notion of a functor or type operators to fully grasp the design pattern. At a glance, the implementation may be confusing, only exacerbated by the several extensions that must be enabled (e.g., `DeriveFunctor`, `TypeOperators`, `FlexibleContexts`).
- *This approach is very elegant, and captures the modularity of our system very well.* The most attractive part of DTALC is its ability to express extensions of preexisting datatypes, even those that are recursive. Regardless, there is one unanswered question: DTALC does not seem to directly support "overriding" of logic. That is, once a function is defined for a constructor, we cannot override the output of that function provided that some other desired feature is enabled.
- *This approach would be maintainable for those with a more advanced understanding of Haskell.* Since DTALC appeals to several advanced features of Haskell, our project may take some time for future maintainers to understand if we choose this approach. Whether complexity is worth it is still an open question.

Superimposition

If we wanted to emphasize extensibility, *superimposition* provides an interesting perspective. Instead of representing expressions as a datatype, we define a systematic way to merge two Haskell and Happy files. For instance, consider the following two definitions (while this is a slightly contrived example).

```
data Expr = EVar String          -- #1
data Expr = EVar Int | EInt Int -- #2
```

If we superimposed `#1` on `#2`, we would produce `data Expr = EVar String | EInt Int`.

The benefits and drawbacks are as follows.

- *This approach is moderately complex*. It requires us to commit time developing the merging semantics, which could instead be applied towards the language structure and error message handling instead.
- *This approach is somewhat elegant*, since it allows us to follow the most "direct" and "natural" encoding of the language in Haskell. It would be good practice to make each file self-contained (that is, it does not refer to definitions not defined in the file).

- *This approach is maintainable in some cases*. The merging semantics may slightly obfuscate how some features interact with others, especially if there are a lot of features and dependencies to consider. However, since our project only deals with a relatively small number of features, this may not be an issue.

Analysis

Overall, I believe that the decision is dependent on what kind of complexity we are willing to accept. I separate these into three categories. *Representational complexity* is the difficulty of understanding how the language is represented. Next, *compositional complexity* is the difficulty of understanding how each part of the language fits together. Finally, *organizational complexity* is the difficulty of propagating changes across features and language tools like the parser, interpreter, and typechecker. A monolithic datatype might increase the complexity here, since every function operating on it would have to be changed in the case of a revision. The expression problem captures each of these notions.

In aggregate, I believe that the team must select a type of complexity they would like to reduce, and then a type of complexity they would be fine with retaining. In my opinion, I find that compositional complexity may be the most unavoidable if we are to *truly* develop a modular implementation. The problem of extension and overriding is inherently difficult (and has been a source of research for many years), so I believe this to be somewhat unavoidable.

Recommendation and Verification

As suggested in the previous section, I find that compositional complexity is unavoidable, so we should not rule out design patterns simply on that basis. With proper documentation and reference to relevant papers, I believe that the accompanying drawbacks can be ameliorated. This means that I would prefer to avoid a monolithic, simple datatype implementation.

In terms of representation, I find that DTALC is much harder to grasp than superimposition. But, DTALC provides us with a Haskell program directly, whilst superimposition requires a kind of preprocessing step. Moreover, it is not clear how Happy files can be superimposed, since productions are not uniquely identified. It might be easier to implement a parser using the DTALC encoding, but this is not immediately clear to me. Nevertheless, I believe the superimposition approach is the most expressive tool while also minimizing the theoretic complexity. *Therefore, I would like to recommend superimposition to begin with as we implement the language.*

To verify our implementation, we have several criteria we can apply. Firstly, we can collectively evaluate the understandability of the implementation. In other words, once we have a workable representation, we should ask everyone whether they could explain the structure of the implementation. If there remains difficulties, the design pattern should be simplified or changed. This is critical as the representation begins to be used in the interpreter, typechecker, and parser. Secondly, we can describe the benefits and drawbacks more concretely once we have started an implementation. I feel that now, it is imperative to begin writing the implementation, and this approach could be an attractive way to begin, if we opt to forgo a much simpler direct implementation of a tiny core calculus.

Constraints

There are no constraints on this decision pertaining to licensing, data use, or confidentiality. **Thus, the most sound decision for our project will and must be derived from collaborative discussion.**

Delegation and Validation

No AI tools were used for this exploration. The use of AI is *strictly* forbidden on our project.